



Kubernetes Architecture

Introduction

Kubernetes (K8s) is a **container orchestration system** designed to **manage, scale, and deploy containerized applications** efficiently.

- What is Kubernetes architecture?
- What are the key components of Kubernetes?
- How does Kubernetes ensure high availability and scalability?

This guide will help you **understand the core architecture of Kubernetes** and its components.

Section 1: Learn

Overview of Kubernetes Architecture

Kubernetes follows a **Master-Worker Node Architecture**, ensuring **high availability and scalability**.

Component	Role
Master Node	Controls and manages the cluster
Worker Node	Runs application workloads
Pods	Smallest deployable unit containing containers
Services	Exposes applications to the network



Component	Role
Controller	Maintains the desired state of the cluster

Master Node Components

The **Master Node** is responsible for **orchestration and cluster management**.

Component	Function
API Server	Handles requests from kubectl and other tools
Controller Manager	Ensures the desired state of the cluster
Scheduler	Assigns workloads (Pods) to Worker Nodes
etcd	Stores cluster state and configurations

Worker Node Components

A **Worker Node** runs the **application workloads** and interacts with the **Master Node**.

Component	Function
Kubelet	Ensures Pods are running on the node
Kube Proxy	Manages networking and load balancing



Component	Function
Container Runtime	Runs the actual containers (Docker, containerd)

Networking in Kubernetes

Kubernetes **networking model** allows Pods to communicate **seamlessly**:

- **Pod-to-Pod Communication** – Uses a flat network model.
- **Pod-to-Service Communication** – Services expose applications.
- **External Access** – Uses Ingress or LoadBalancer services.

Interesting Fact

Kubernetes was designed to **scale horizontally**, meaning more worker nodes can be added dynamically **without downtime**.

Section 2: Practice

1. Checking Cluster Architecture

To see the cluster components, run:

```
kubectl cluster-info
```

To list nodes in the cluster:

```
kubectl get nodes
```

2. Inspecting Master and Worker Node Components

To check the running **Pods in the master node**:

```
kubectl get pods -n kube-system
```

To check running **Pods on worker nodes**:



```
kubectl describe node <worker-node-name>
```

3. Deploying a Sample Application

Step 1: Create a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Apply the deployment:



```
kubectl apply -f nginx-deployment.yaml
```

Step 2: Check Running Pods

```
kubectl get pods -o wide
```

Step 3: Expose the Deployment

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80
```

Find the assigned port:

```
kubectl get services
```

Access the application at **http://localhost:**.

Section 3: Know More

Frequently Asked Questions

1. **What is the role of `etcd` in Kubernetes?**
 - It is a distributed key-value store that stores cluster configuration and state.
2. **What is the difference between a Pod and a Node?**
 - A **Pod** is a unit of deployment that runs one or more containers, while a **Node** is a machine that runs Pods.
3. **How does Kubernetes handle networking?**
 - Kubernetes assigns each **Pod** a **unique IP address**, enabling seamless communication.
4. **What happens if a worker node fails?**



- The **Control Plane** detects failure and reschedules Pods to healthy nodes.
5. **Can Kubernetes run without Docker?**
- Yes, Kubernetes supports **other container runtimes** like **containerd** and **CRI-O**.
-

Conclusion

Kubernetes follows a **distributed architecture** that enables **scalability, high availability, and automation**.

Understanding the **Master-Worker Node model** helps in managing and troubleshooting Kubernetes clusters efficiently.